

Laboratorium przetwarzanie obrazów

Radosław Piotrowicz

Jak rozszerzyć program o nowe algorytmy

Spis treści

1	Gdzie znajdują się interesujące nas rzeczy.	3
2	Algorithms.hpp	3
3	App.cpp	5
4	<nazwa algorytmu>.hpp	7
5	<nazwa algorytmu>.cpp	8
5.1	Wersja minimum	8
5.2	Wersja pełna	9
5.3	Parametry	9
5.3.1	Pojedynczy parametr.	10
5.3.2	Kilka parametrów	10
5.3.3	Parametry tabelkowe	12
6	ImGui	16
6.1	ImGuiDemo	16
6.2	Link do biblioteki	18

1 Gdzie znajdują się interesujące nas rzeczy.

Jeżeli chcemy dodać nowy algorytm, należy utworzyć 2 nowe pliki oraz zmodyfikować 2 istniejące:

- **Algorithms.hpp** dodanie dyrektywy `#include` z plikiem nagłówkowym nowego algorytmu
- **App.cpp** dodanie do tablicy (vectora) dostępnych algorytmów nowego algorytmu (W metodzie `Init` okolicie liniiki 30).
- **<nazwa algorytmu>.hpp** stworzenie nowej klasy dziedziczącej po `Algorithm` oraz nadpisanie metod.
- **<nazwa algorytmu>.cpp** implementacja metod.

2 Algorithms.hpp

```
#ifndef ALGORITHMS_HPP
#define ALGORITHMS_HPP

// NEW ALGORITHMS HEADERS HERE

#include "Negative.hpp"
#include "Brighten.hpp"
#include "Contrast.hpp"
#include "Exponentiation.hpp"
#include "Logarithm.hpp"
#include "LeveledHistogram.hpp"
#include "Masking.hpp"
#include "Mixing.hpp"
#include "Binarization.hpp"
#include "LinearFilter.hpp"
#include "MedianFilter.hpp"
#include "Erosion.hpp"
#include "Dilatation.hpp"
#include "Opening.hpp"
#include "Closing.hpp"
#include "ContourInner.hpp"
#include "ContourOuter.hpp"
#include "Skeletonization.hpp"
#include "Hough.hpp"
#include "ExampleAlg.hpp"

#endif
```

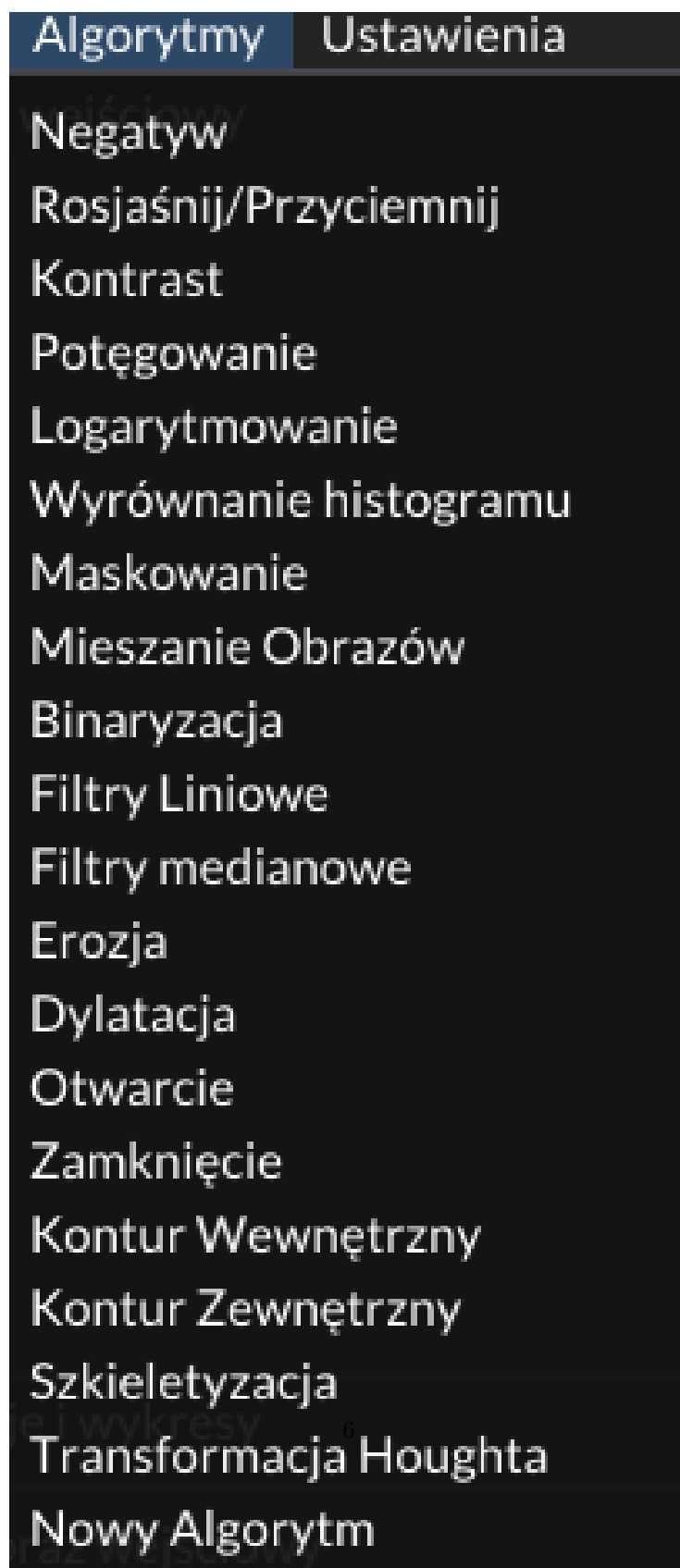
Rysunek 1: Plik nagłówkowy z algorytmami

3 App.cpp

```
... // HERE ADD NEW ALGS TO THE VECTOR
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Negative());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Brighten());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Contrast());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Exponentiation());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Logarithm());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new LeveledHistogram());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Masking());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Mixing());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Binarization());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new LinearFilter());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new MedianFilter());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Erosion());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Dilatation());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Opening());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Closing());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new ContourInner());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new ContourOuter());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Skeletonization());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new Hough());
... algorithmsAvailable.emplace(algorithmsAvailable.end(), new ExampleAlg());
```

Rysunek 2: Sekcja dodawania algorytmów do tablicy

Kolejność jest od góry do dołu.



Rysunek 3: Menu wyboru algorytmów

4 <nazwa algorytmu>.hpp

Należy utworzyć nowy plik w katalogu `./include/algorithms`. Przykładowy nowy algorytm jest przedstawiony na rysunku 4. Metody od zapisu i wczytania są **opcjonalne!**.

```

#ifndef EXAMPLEALG_HPP
#define EXAMPLEALG_HPP

#include "Algorithm.hpp"

class ExampleAlg : public Algorithm
{
public:
    ... ExampleAlg();

    ... void ParamsMenu() override;
    ... void AlgorithmFunction(Image *outputImage) override;
    ... void ResetToDefaults() override;

    ... void Save(std::ofstream &file) override; // opcjonalne
    ... void Load(std::ifstream &file) override; // opcjonalne

private:
    ... int exampleParam = 0;
};

#endif

```

Rysunek 4: Przykładowy plik nagłówkowy nowego algorytmu

Można też utworzyć klasę pośrednią, jeżeli kilka algorytmów będzie miało takie same parametry. Przykładem jest **MorphologicAlgorithm.hpp**.

5 <nazwa algorytmu>.cpp

Tutaj jest najwięcej do zrobienia.

5.1 Wersja minimum

Na rysunku 5 przedstawiono minimum co trzeba zrobić, aby algorytm zadziałał.

```

#include "ExampleAlg.hpp"

// WERSJA MINIMUM

ExampleAlg::ExampleAlg() { algorithmName = "Nowy Algorytm"; }

void ExampleAlg::ParamsMenu() {} // może być puste (tylko parametry trzeba wtedy ustawić w kodzie)

void ExampleAlg::AlgorithmFunction(Image *outputImage)
{
    ... CopyToLocalVariable(outputImage);
    ... // tutaj kod
    ... SaveToOutput(outputImage);
}

void ExampleAlg::ResetToDefaults() {} // może być puste (tylko parametry się nie zresetują do wartości domyślnej)

void ExampleAlg::Save(std::ofstream &file) {} // można nie zrobić nic
void ExampleAlg::Load(std::ifstream &file) {} // można nie zrobić nic

```

Rysunek 5: Minimalna implementacja

5.2 Wersja pełna

Na rysunku 6 przedstawiono przykładową implementację wszystkich metod.

```
#include "ExampleAlg.hpp"
#define PI 3.14
#define DRZWI 2137

ExampleAlg::ExampleAlg() { algorithmName = "Nowy Algorytm"; }

void ExampleAlg::ParamsMenu() { ImGui::SliderInt("0 ile rozjaśnić/przyciemnić?", &exampleParam, -255, 255); }

void ExampleAlg::AlgorithmFunction(Image *outputImage)
{
    CopyToLocalVariable(outputImage);
    for (int row = 0; row < copy.GetHeight(); row++)
    {
        for (int col = 0; col < copy.GetWidth(); col++)
        {
            auto pix = copy.GetPixel(col, row);
            pix.r += exampleParam * PI * DRZWI;
            pix.g += exampleParam * PI * DRZWI;
            pix.b += exampleParam * PI * DRZWI;
            copy.SetPixel(col, row, pix);
        }
        // opcjonalne
        if (Canceled(outputImage))
            return;
        AutomaticRefresh(outputImage);
    }
    SaveToOutput(outputImage);
}

void ExampleAlg::ResetToDefaults() { exampleParam = 0; }

void ExampleAlg::Save(std::ofstream &file) { file << "ExampleParam" << CONFIG_SPLIT_CHAR_A << exampleParam << std::endl; }

void ExampleAlg::Load(std::ifstream &file)
{
    try
    {
        exampleParam = std::stoi(SplitLine(file));
    }
    // jeżeli wysąpi błąd to ustawić na domyślne
    catch (const std::exception &e)
    {
        exampleParam = 0;
    }
}
```

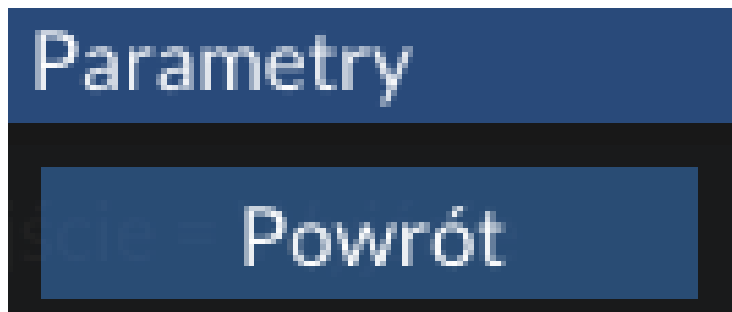
Rysunek 6: Pełna implementacja

Ważne

- Automatyczne odświeżanie oraz Anulowanie są opcjonalne.
- Można posiłkować się metodami od parametrów, resetu, zapisu i odczytu z innych algorytmów jako przykładów (powinno tam być sporo przypadków).

5.3 Parametry

Tutaj jest dodatkowy opis oraz kilka przykładów graficznego menu do ustawienia parametrów. Bazowo (jeżeli zostawimy metodę od parametrów pustą) okienko będzie wyglądać jak na rysunku 7.



Rysunek 7: Puste

Jeżeli jednak chcemy dorobić menu graficzne poniżej jest przedstawione kilka przykładów:

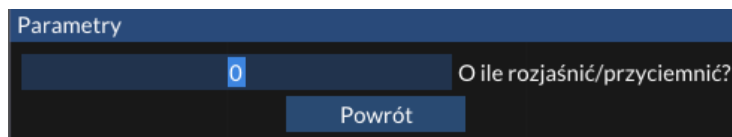
5.3.1 Pojedynczy parametr.

Dla niektórych algorytmów wystarcza obsługa tylko jednego parametru na przykład:

ImGui::SliderInt("O ile rozjaśnić/przyciemnić?", ¶ms.value, -255, 255)

Najpierw jest opis slidera, potem wartość, do której zapisany będzie wynik, następnie ograniczenia dolne i górne.

A tak będzie wyglądało okienko.



Rysunek 8: Pojedynczy parametr

5.3.2 Kilka parametrów

Czasami może być potrzebne kilka parametrów do algorytmu. Takich jak rodzaj metody lub liczba progów. Wtedy można to zrobić w ten sposób.

```

void Binarization::ParamsMenu()
{
    // set if auto or manual
    ImGui::Text("Wybierz czy chcesz ustawić samemu\nczy automatycznie");
    ImGui::RadioButton("Ręcznie ustaw próg", &method, None);
    ImGui::SameLine();
    ImGui::RadioButton("Metoda Gradientowa", &method, Gradient);
    ImGui::SameLine();
    ImGui::RadioButton("Metoda iteracyjna", &method, Iteration);
    if (method == None)
    {
        ImGui::Separator();
        ImGui::Text("Wybierz ilość progów");
        ImGui::RadioButton("Jeden próg", &boundCount, 1);
        ImGui::SameLine();
        ImGui::RadioButton("Dwa progi", &boundCount, 2);
        ImGui::Separator();
        ImGui::Text("Ustaw progi");
        ImGui::SliderInt("t", &lowerBound, 0, 255);
        if (boundCount == 2)
        {
            ImGui::SliderInt("t1", &upperBound, 0, 255);
        }
    }
}

```

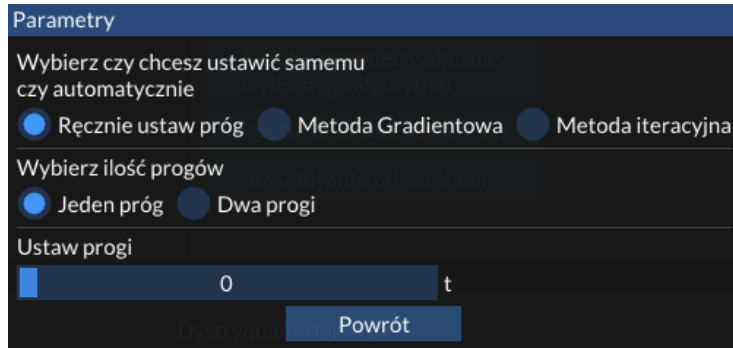
Rysunek 9: Kilka parametrów

Na początku metody pomocniczej znajduje się pierwszy wybór (metody), obsługiwany jest przez funkcję **ImGui::RadioButton()**. Funkcja ta przyjmuje jako parametry:

- **const char *label** nazwa opcji
- **int *v** wskaźnik do zmiennej z obecnie wybranym stanem
- **int v_button** numer stanu przypisany do opcji (najlepiej zdefiniowany w enum)

ImGui::SameLine() informuje, żeby umieścić elementy w tej samej linii. **ImGui::Separator()** tworzy linię separującą elementy. Następnie znajduje się kolejny wybór (ilości progów), zrobiony analogicznie do wyboru metody. Jednak pojawi się on tylko wtedy gdy ustawiamy progi ręcznie. Również, jeżeli ustawiamy progi ręcznie pojawią się dwa slidery do wyboru wartości progów.

Okienko będzie wyglądało tak:



Rysunek 10: Okienko kilku parametrów

5.3.3 Parametry tabelkowe

Przy niektórych algorytmach parametrem jest macierz (maska albo element strukturalny).

```
void LinearFilter::DrawLinearDisplayArray()
{
    ImGui::SetCursorPosX(ImGui::GetWindowWidth() / 2 - (ARRAY_FIELD_WIDTH * linearFilterSize) / 2);
    if (ImGui::BeginTable("Maska", linearFilterSize, ImGuiTableFlags_Borders, ImVec2(ARRAY_FIELD_WIDTH * linearFilterSize, 0)))
    {
        for (int i = 0; i < linearFilterSize; i++)
        {
            ImGui::TableSetupColumn("#", ImGuiTableColumnFlags_WidthFixed, ARRAY_ITEM_WIDTH);
            for (int row = 0; row < linearFilterSize; row++)
            {
                ImGui::TableNextRow();
                for (int col = 0; col < linearFilterSize; col++)
                {
                    ImGui::TableSetColumnIndex(col);
                    if (linearFilterSize == 53x3)
                        ImGui::Text("%d", linearMask3x3[row][col]);
                    else if (linearFilterSize == 53x5)
                        ImGui::Text("%d", linearMask5x5[row][col]);
                    else
                        ImGui::Text("%d", linearMask7x7[row][col]);
                }
            }
        }
        ImGui::EndTable();
    }
}
```

Rysunek 11: Tabelka w kodzie

Aby rozpocząć tabelkę, należy użyć funkcji `ImGui::BeginTable()`, i musi się ona znajdować w instrukcji warunkowej. Funkcja ta przyjmuje następujące parametry:

- `const char *str_id` nazwa tablicy
- `int columns` ilość kolumn
- `ImGuiTableFlags flags` flagi dotyczące wyglądu tablicy
- `const ImVec2 &outer_size` rozmiar tablicy
- `float inner_width` wewnętrzna szerokość komórek

W naszym przypadku interesować będą nas pierwsze 3-4 parametry.

Następnie należy w pętli podwójnej utworzyć wszystkie elementy.

W zewnętrznej pętli (dla wierszy) używamy `ImGui::TableNextRow()` aby przejść do kolejnego wiersza. Następnie w wewnętrznej pętli (dla kolumn) ustawiamy kursor na kolumnę przy użyciu `ImGui::TableSetColumnIndex(col)` i dodajemy element.

Aby dodać nowy element, można użyć różnych elementów na przykład:

- `ImGui::Text()` - do wyświetlenia elementu bez możliwości wprowadzenia.
- `ImGui::InputInt()` - do wprowadzenia liczby całkowitej z powiązaniem do elementu o odpowiednim indeksie.
- `ImGui::Checkbox()` - do oznaczenia elementu, którego wartości mogą przyjmować `true/false`.

Tak będą wyglądać tablice w kolejności takiej, jak na liście:

Parametry

Wybierz rozmiar filtra

☐ 3x3
 ☐ 5x5
 ☒ 7x7

Wybierz rodzaj filtra

☐ Uśredniający
 ☒ Gauss
 ☐ Sobel Poziomy
☐ Sobel Pionowy
 ☐ Laplasjan
 ☐ Wyostrzający
☐ Własna

0	0	1	2	1	0	0
0	3	13	22	13	3	0
1	13	59	97	59	13	1
2	22	97	159	97	22	2
1	13	59	97	59	13	1
0	3	13	22	13	3	0
0	0	1	2	1	0	0

Powrót

Rysunek 12: Tablica z samym wyświetlaniem zawartości

Parametry

Wybierz rozmiar filtra

☐ 3x3
 ☐ 5x5
 ☒ 7x7

Wybierz rodzaj filtra

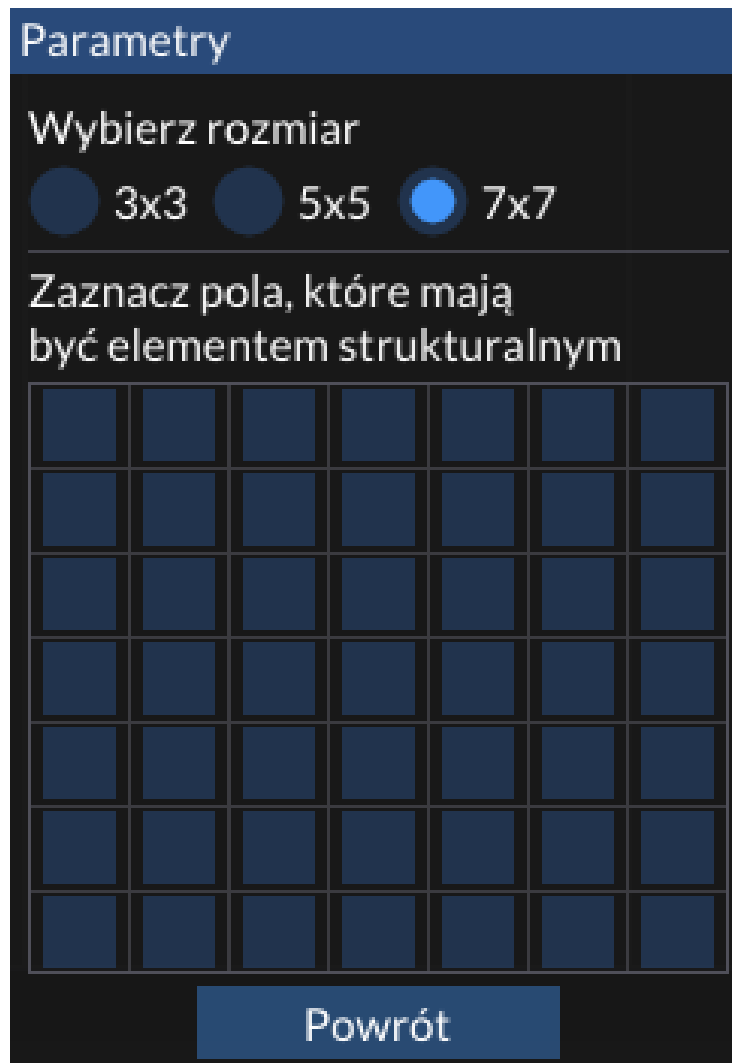
☐ Uśredniający
 ☐ Gauss
 ☐ Sobel Poziomy
☐ Sobel Pionowy
 ☐ Laplasjan
 ☐ Wyostrzający
☒ Własna

Ustaw własną maskę

0	-	+	0	-	+	1	-	+	2	-	+	1	-	+	0	-	+	0	-	+
0	-	+	3	-	+	13	-	+	22	-	+	13	-	+	3	-	+	0	-	+
1	-	+	13	-	+	59	-	+	97	-	+	59	-	+	13	-	+	1	-	+
2	-	+	22	-	+	97	-	+	159	-	+	97	-	+	22	-	+	2	-	+
1	-	+	13	-	+	59	-	+	97	-	+	59	-	+	13	-	+	1	-	+
0	-	+	3	-	+	13	-	+	22	-	+	13	-	+	3	-	+	0	-	+
0	-	+	0	-	+	1	-	+	2	-	+	1	-	+	0	-	+	0	-	+

Powrót

Rysunek 13: Tablica z wprowadzaniem liczb całkowitych



Rysunek 14: Tablica z checkboxami

Na koniec tablice należy zakończyć używając `ImGui::EndTable()`.

Dodatkowe uwagi:

- Jeżeli tablica ma służyć do wprowadzania danych należy użyć `ImGui::PushID(nr wiersza)` - przed `ImGui::TableNextRow()` i `ImGui::PopID()` - po wewnętrznej pętli.
- Jeżeli ustawiamy ID należy najpierw użyć `std::string s = "##" + std::to_string(col)`, aby ustalić to ID, a następnie przekazać je do funkcji od elementu na przykład `ImGui::Checkbox(s.c_str(), &a3x3[row][col])`.

- Jeżeli nie potrzebujemy opisów kolumn, można użyć **ImGui::TableSetupColumn("##")** dla każdej kolumny.
- Jeżeli chcemy ustawić szerokość elementu użyjemy **ImGui::TableSetupColumn("##", ImGuiTableColumnFlags_WidthFixed, ARRAY_ITEM_WIDTH)**. Tablica powinna mieć podany stały rozmiar w **ImGui::BeginTable()**.
- Jeżeli tablica nie ma ustawionego rozmiaru na początku można określić szerokość elementu możemy użyć **ImGui::PushItemWidth(ARRAY_INPUT_WIDTH)**.
- Przykłady metod obsługujących tabelki można znaleźć w plikach **LinearFilter.cpp**, **MedianFiler.cpp**, **MorphologicAlgorithm.cpp**.

6 ImGui

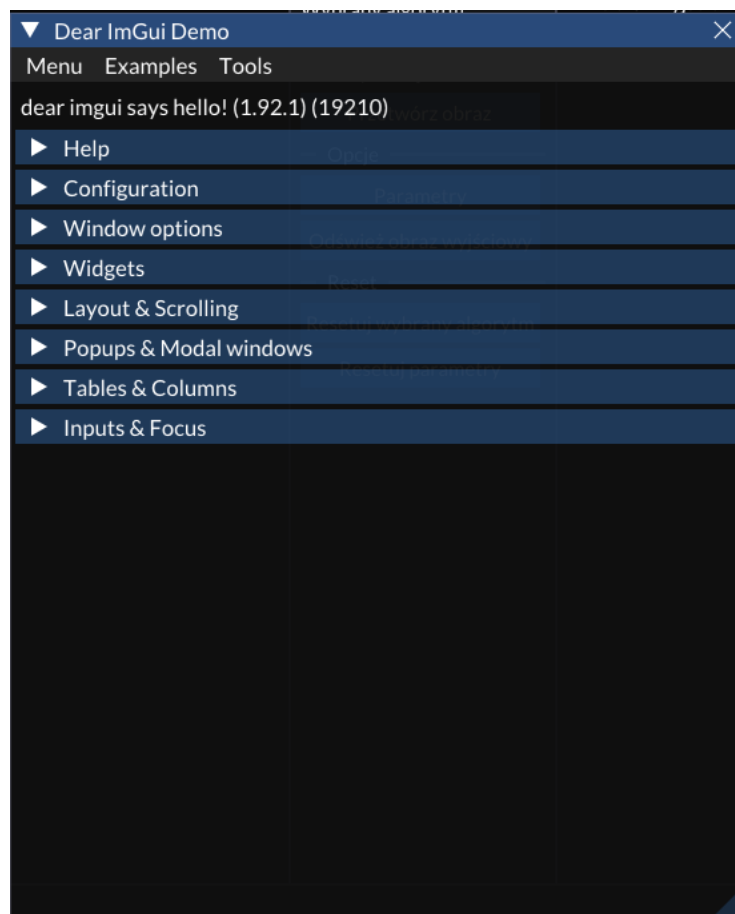
W tej sekcji opisze przydatne informacje dotyczące użytej biblioteki graficznej.

6.1 ImGuiDemo

Do programu dołączone jest ImGuiDemo, które zawiera przykłady możliwości biblioteki graficznej.

Znajduje się ono w sekcji **Pomoc->Pokaż ImGuiDemo**

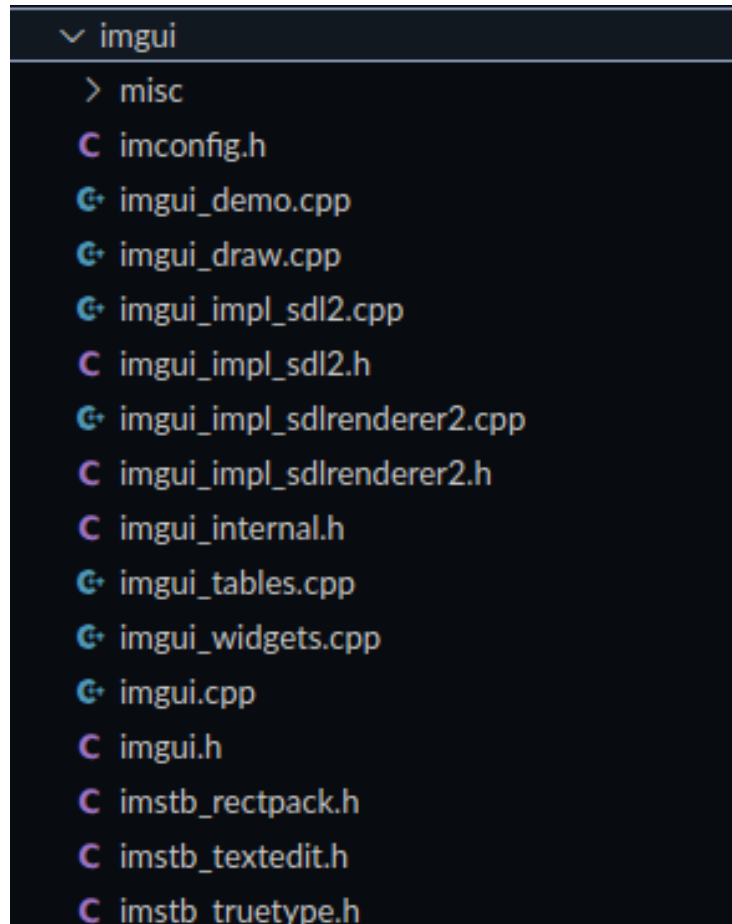
Okno wygląda tak:



Rysunek 15: Okno ImGuiDemo

Przykłady w kodzie znajdują się w pliku **imgui_demo.cpp**. Plik ten znajduje się w folderze ImGui.

W tym pliku powinno znajdować się wszystko, co może być potrzebne przy zapoznawaniu się z biblioteką. Ponadto twórcy ImGui zachęcają do zachowania tego pliku w projektach w celu posiadania przykładów implementacji funkcjonalności.



Rysunek 16: Folder ImGui

6.2 Link do biblioteki

Na końcu zostawiam link do Githuba, z biblioteką. Znajdują się tam wszystkie potrzebne informacje.

Github: <https://github.com/ocornut/imgui>